

# 0.69.3. Chercher le A sur l'écran : lire — en simple

Le 0.69 en simple : la croix en une ligne ; la recherche avec lire() reste, c'est elle qu'on apprend ici.

## 1. Ce qu'il faut comprendre

C'est le 0.69, en plus simple : la croix est remplacée par `deplace_croix`.

La recherche, elle, reste écrite à la main : c'est ce que la leçon apprend, `lire()` et les deux boucles. Simplifier, ce n'est pas tout cacher : on ne cache que ce qu'on sait déjà faire.

### Les fonctions de cette leçon — et le tuto de chacune

- `#include <deplace_croix>` → [0.66.3. La fonction deplace\\_croix\(\) — suivre la croix, case par case](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <lire>` → [0.68.4. La fonction lire\(\) — lire une case](#)

## 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1328 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      79 o ← la console, à l'allumage - la mise en route de la console
deplace_croix() 217 o ← main() - une fonction fournie par le langage
main()          140 o ← démarrage - ta fonction
VBlank          8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage   28 o ← image() - attend l'image suivante
AttendreVBlank  16 o ← démarrage - attend que l'écran soit libre
AttendreAcces   12 o ← EffacerCases, EcrireNombre - attend de pouvoir écrire une case
AdresseCase     10 o ← effacer(), poser(), lire() - trouve l'adresse d'une case de l'écran
EffacerCases    12 o ← effacer() - efface des cases
EcrireNombre     33 o ← nombre() - écrit un nombre en chiffres
CopierTuiles    18 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte    14 o ← démarrage - vide les deux cartes de l'écran
LireManette     37 o ← bouton() - lit les huit boutons
Tuiles          704 o ← CopierTuiles - la police entière
```

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

```
nombre()      x 4    40 o sur place, dans main() + EcrireNombre (33 o)
bouton()      x 5    25 o sur place, dans deplace_croix(), main() + LireManette (37 o)
effacer()     x 1    15 o sur place, dans deplace_croix() + AdresseCase (10 o), EffacerCases (12 o)
poser()       x 1    13 o sur place, dans deplace_croix() + AdresseCase (10 o)
```

```

lire()          x 1    11 o sur place, dans main() + AdresseCase (10 o)
image()         x 1    3 o sur place, dans main() + AttendreImage (28 o)
images()        x 1    2 o sur place, dans deplace_croix()
Tes fonctions appelées :
deplace_croix() x 1    19 o sur place, dans main()

```

## 1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// x, y, trouveX, trouveY : voir l'onglet variables.h

#include <deplace_croix> // une tuile qui suit la croix, case par case
#include <ALPHABET>      // les lettres de la police : ALPHABET[0] est le A
#include <nombre>         // écrit un nombre en chiffres
#include <bouton>          // lit un bouton de la manette
#include <lire>            // lit la tuile posée sur une case
#include "variables.h"

```

## 2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```

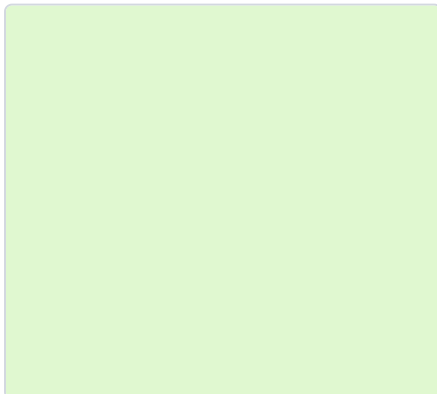
int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 133);
        nombre(0, 17, x);
        nombre(4, 17, y);

        if (bouton(A)) { // on cherche le A sur l'écran
            for (uint8_t l = 0; l < 18; l++) {
                for (uint8_t c = 0; c < 20; c++) {
                    if (lire(c, l) == ALPHABET[0]) {
                        trouveX = c;
                        trouveY = l;
                    }
                }
            }
            nombre(10, 17, trouveX);
            nombre(14, 17, trouveY);
        }
    }
}

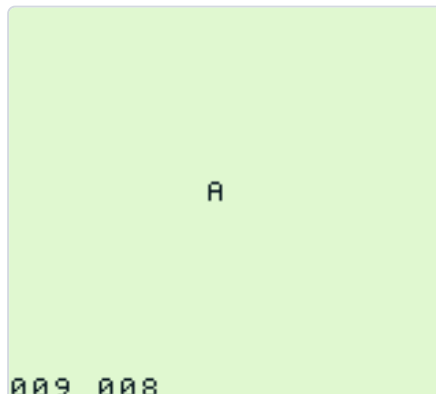
```

## 3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de commencer — image 1

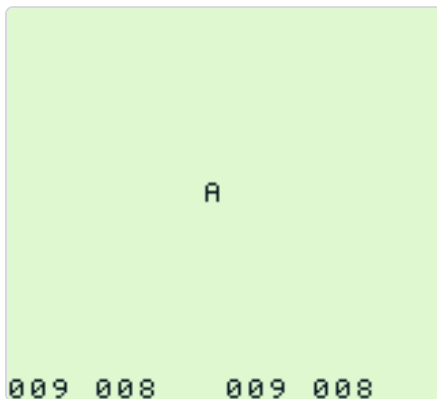


un tiers de seconde plus tard : le premier dessin est posé — image 20

## 4. Ce que les touches changent

---

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



A — l'écran a changé

## 5. Ce qu'on doit voir

---

Comme au 0.69 : avec A, la position trouvée s'affiche à droite, la même qu'à gauche.

## 6. Ce que la console répond

---

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ lire() a trouvé le A

---

Ensuite : **0.69.4. La fonction `sprite()` — un lutin au pixel près** — chapitre 0 : Avant tout. · [gameboy3](#) — écrire une cartouche Game Boy en C++